Lab02Tarefa03 – Algoritmo K-means

Nome: Leonardo Scholl Sternberg

Curso: Algoritmos para Agrupamento de Dados

Especialização em Inteligência Artificial para Engenheiros

Avaliação do Código

1: Importação de Bibliotecas

```
# Importa as bibliotecas necessárias
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import numpy as np
```

Este bloco inicial configura o ambiente de trabalho, carregando as ferramentas necessárias para a manipulação e visualização dos dados:

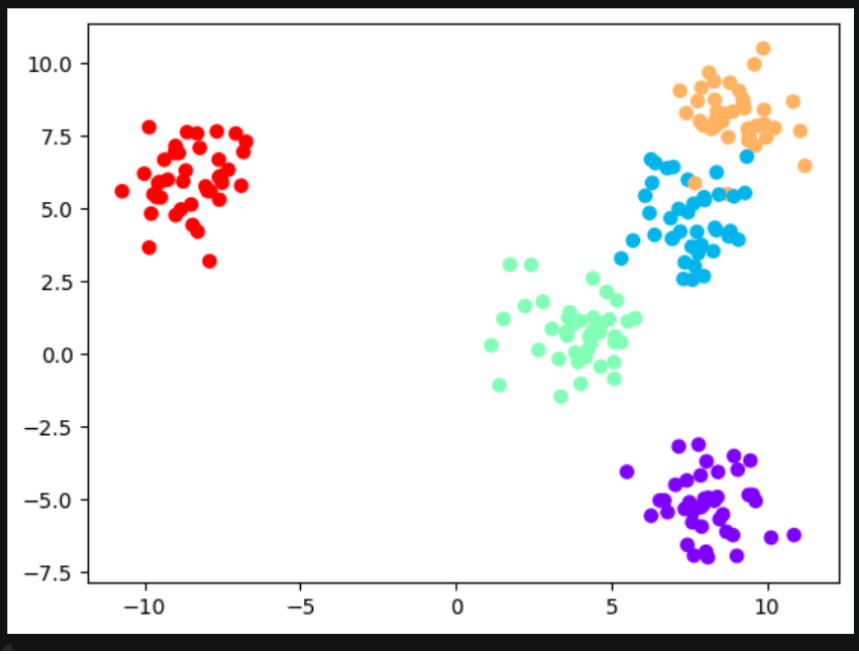
- **Pandas:** Responsável pelo carregamento e estruturação da base de dados.
- **Matplotlib:** Utilizada para a criação dos gráficos de dispersão (blobs) e da visualização da curva do WCSS (Método do Cotovelo).
- **NumPy:** Utilizada para o processamento numérico e cálculos matemáticos das distâncias entre os dados e os centroides.

2: Geração e Visualização de Dados

```
from sklearn.datasets import make_blobs

# criando dados
data, labels = make_blobs(n_samples=200, n_features=2, centers=5, cluster_std=1, random_state=150)
plt.scatter(data[:,0], data[:,1], c=labels,cmap='rainbow')
```

```
<matplotlib.collections.PathCollection at 0x78291a989610>
```

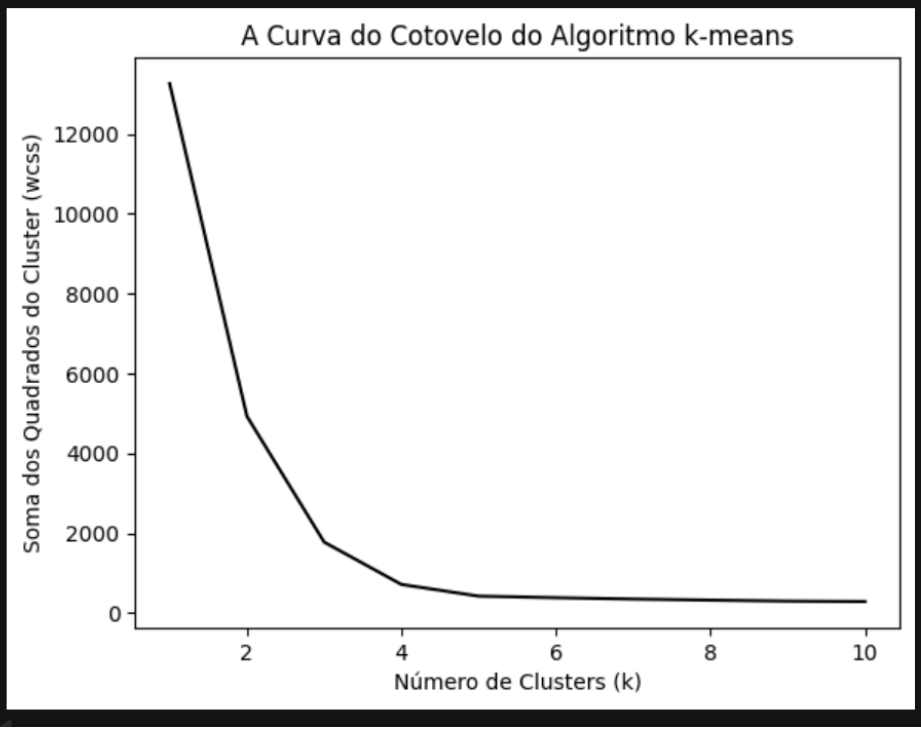


Este trecho utiliza a biblioteca Scikit-Learn para criar o conjunto de dados sintético que será base para o agrupamento:

- **make_blobs**: Função que gera agrupamentos de pontos (*clusters*) com distribuição gaussiana. No código, são geradas 200 amostras bidimensionais divididas em 5 centros distintos.
- **plt.scatter**: Gera o gráfico de dispersão inicial. Os dados são coloridos de acordo com os rótulos originais (c=labels) usando o mapa de cores 'rainbow', permitindo visualizar a separação espacial clara entre os grupos antes da aplicação do algoritmo K-Means.

3: Determinação do K Ideal (Método do Cotovelo)

```
from sklearn.cluster import KMeans
wcss = []
for i in range (1,11):
    kmeans = KMeans(n_clusters = i, init = 'k-means++', random_state = 150)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss, color='k')
plt.title('A Curva do Cotovelo do Algoritmo k-means')
plt.xlabel('Número de Clusters (k)')
plt.ylabel('Soma dos Quadrados do Cluster (wcss)')
plt.show()
```



Este bloco de código executa a análise iterativa para encontrar o número ótimo de clusters:

- **Loop de Treinamento:** O algoritmo K-Means é executado 10 vezes (de k=1 a k=10). O parâmetro init='k-means++' é utilizado para otimizar a convergência inicial dos centroides.
- **Cálculo do WCSS:** Para cada iteração, o valor da inércia (soma dos quadrados das distâncias internas) é armazenado na lista wcss.
- **Curva do Cotovelo:** O gráfico gerado correlaciona o número de clusters com o erro (WCSS). A análise visual indica que o "cotovelo" ocorre em k=5, pois após esse ponto a redução do erro torna-se marginal, definindo este como o valor ideal para o treinamento final.

4: Treinamento do Modelo K-Means

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=5, init='k-means++')
kmeans.fit(data)
```

```
KMeans
KMeans(n_clusters=5)
```

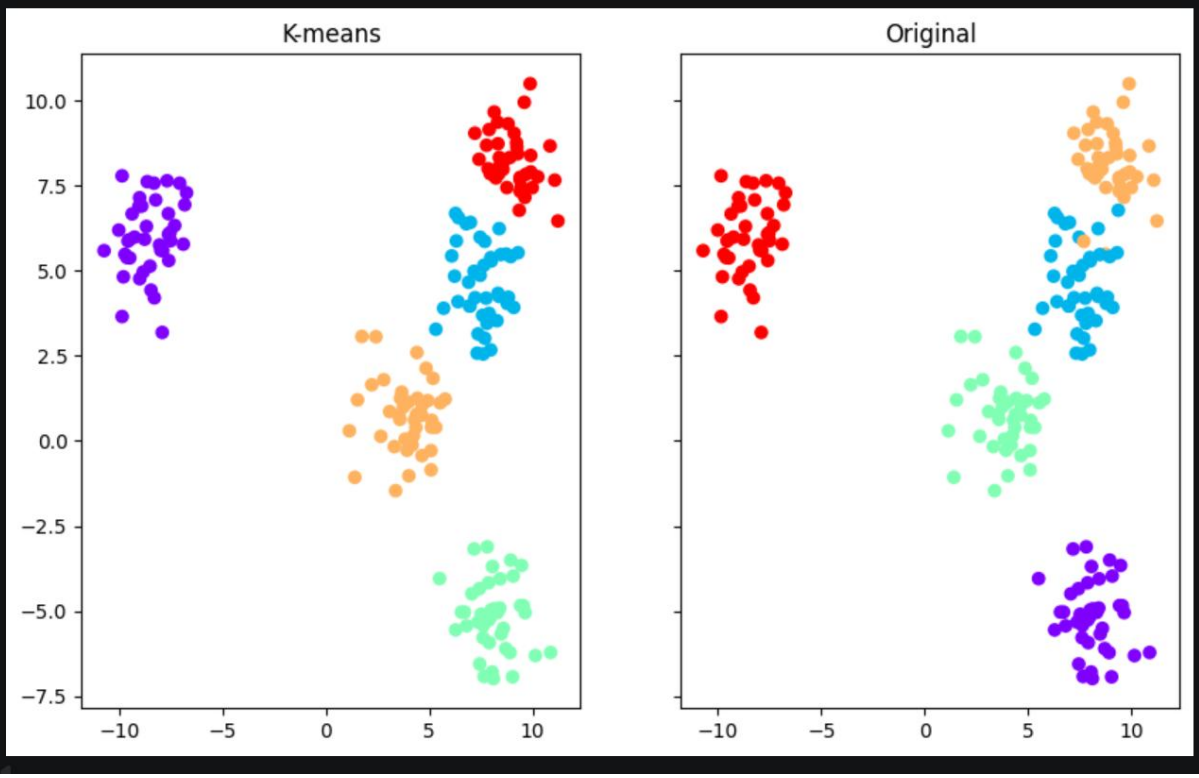
Nesta etapa, o algoritmo é executado com os parâmetros definidos pela análise anterior:

- **Definição do Parâmetro k:** O modelo é instanciado com n_clusters=5, valor identificado como ideal através do Método do Cotovelo para representar os agrupamentos naturais dos dados.
- **Inicialização Inteligente:** Utiliza-se init='k-means++', uma técnica que seleciona os centroides iniciais de forma estratégica para acelerar a convergência e evitar mínimos locais.
- **Treinamento (fit):** O método kmeans.fit(data) executa o processo iterativo de ajuste, calculando as coordenadas finais dos 5 centroides e atribuindo cada ponto da base ao cluster mais próximo.

5: Comparação de Resultados

```
f, (ax1, ax2) = plt.subplots(1, 2, sharey=True, figsize=(10,6))
ax1.set_title('K-means')
ax1.scatter(data[:,0],data[:,1],c=kmeans.labels_,cmap='rainbow')
ax2.set_title('Original')
ax2.scatter(data[:,0],data[:,1],c=labels,cmap='rainbow')
```

```
<matplotlib.collections.PathCollection at 0x782917146d50>
```



O trecho final do código realiza uma análise comparativa visual entre os dados originais e o agrupamento realizado pelo modelo:

- **Subplots:** Utiliza plt.subplots(1, 2) para exibir dois gráficos lado a lado, facilitando a validação direta do desempenho do algoritmo.
- **Gráfico K-means:** Exibe a classificação feita pelo modelo (kmeans.labels_). É possível observar que o algoritmo identificou corretamente os 5 agrupamentos, atribuindo cores distintas a cada cluster encontrado.
- **Gráfico Original:** Apresenta a distribuição original dos dados. A similaridade visual entre ambos os gráficos confirma que o treinamento foi bem-sucedido e que o valor de k=5 foi a escolha correta para representar a estrutura real do conjunto de dados.

6: Coordenadas dos Centroides

```
kmeans.cluster_centers_
```

```
array([[-8.53502186,  5.98022091],
       [ 7.51724497,  4.61232782],
       [ 8.1037214 , -5.1507853 ],
       [ 3.89242402,  0.72606842],
       [ 9.01735573,  8.23432609]])
```

O comando kmeans.cluster_centers_ retorna as posições finais dos centros de cada agrupamento após a convergência do algoritmo:

- **Representação Espacial:** O resultado é um array bidimensional onde cada linha contém as coordenadas (x, y) do centro geográfico de um dos 5 clusters identificados.
- **Significado Técnico:** Estes pontos representam os "protótipos" de cada grupo, sendo os locais onde a soma das distâncias ao quadrado para todos os pontos do respectivo cluster é minimizada.
- **Utilidade:** Essas coordenadas são fundamentais para classificar novos dados (atribuindo-os ao centroide mais próximo) e para entender o comportamento médio de cada segmento encontrado pelo modelo.

Discussão e Considerações Finais

O código aplica o algoritmo K-Means em um conjunto de dados do tipo *blobs* para identificar agrupamentos naturais. Através do Método do Cotovelo, o script calcula o WCSS (Soma dos Quadrados Dentro dos Clusters) para diferentes valores de k, permitindo identificar visualmente que o número ideal de grupos é 5, ponto onde a redução do erro se estabiliza.

O treinamento final com k=5 e a inicialização k-means++ permitiu mapear com precisão as coordenadas dos centroides e classificar cada ponto da amostra. A comparação entre os dados originais e o resultado do modelo confirma que a técnica foi eficaz na segmentação dos dados, validando o uso do WCSS como critério matemático robusto para a definição de clusters em aprendizado não supervisionado.